

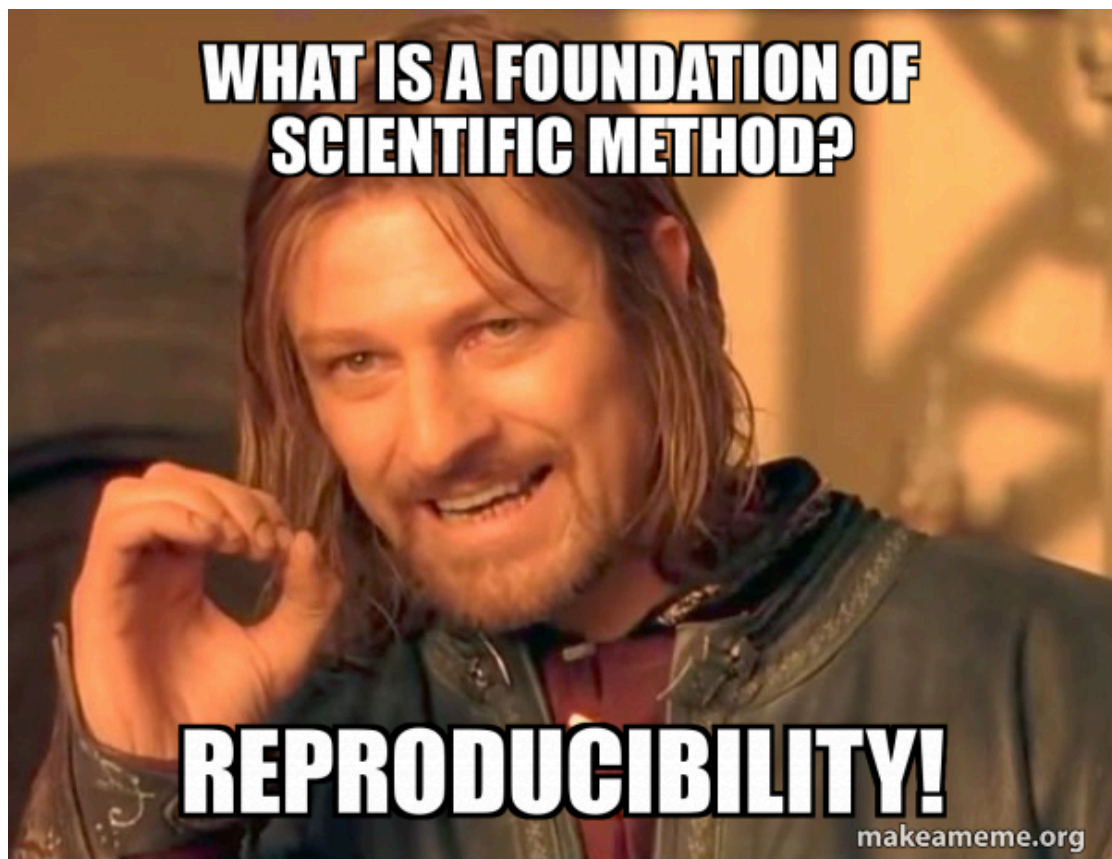
Kernel: Python 3 (system-wide)

Reproducibility, the BASH console and GIT

Welcome to the 7th week of classes! Congratulations, you've made it through data analysis, visualizations and how to give and critique reports! Today, we will be going through the topics of reproducibility and how you can use git to help organize and maintain your files

Reproducibility: What is it, What does it mean, Why it's important

The concept of reproducibility is just that - **is your work reproducible?** Reproduction of methods, analyses and results is important not only in scientific research but also in day-to-day activities. It is a major factor that determines if **research results are consistent and valid** using the same methods used in the original study. Many research studies provide significant results but their findings cannot be reproduced. It raises the question of whether the research question or the hypothesis is valid. If your work is reproducible, then **valid experiments and results build authentic discoveries that are more reliable** in your field and help develop robust literature for future research studies. While reproducibility ensures that your work is accurate and reliable, **and encourages community involvement and participation**. *Remember, if your foundation is shaky then so is all the work built upon it.*



There are different fields of reproducibility :

1. **Scientific** Reproducibility - **Can your results be reproduced from your methods?**
2. **Computational** Reproducibility - **Can your results be reproduced by your code?**

3. **Statistical** Reproducibility - **Are your methods robust enough to reproduce your results?**

Statistical reproducibility requires **formal training** in study design and interpretation. I.e taking a sample of the population, randomizing treatments, having a large number of replicates, etc – this is part of the methodological reproducibility and statistical reproducibility.

While talk of reproducibility is all good and dandy, it may not be available to everyone. There are significant barriers to reproducibility including:

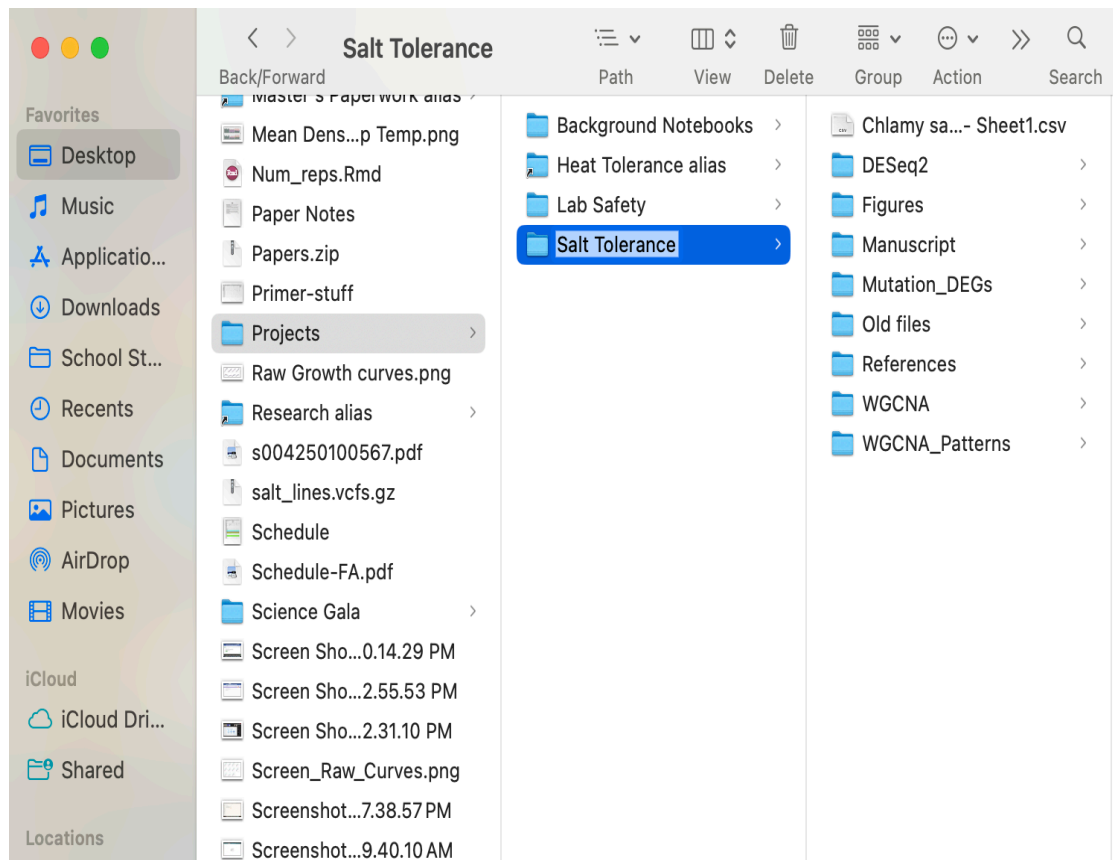
- **Cost**
- Structure of dissemination does not currently fully support reproducibility
- **Lack of training and incentives**
- **Takes time, held to a higher standard**
- Privacy and security
- Differences between fields

Your job as a scientist and a researcher is to try your best to ensure that your work is reproducible. **If the ability is beyond your means, do not hesitate to say so.** Science is about being open and transparent

There are many tools and conventions to help with your reproducibility journey. These include having a proper workflow and file organization system, as well as GIT for version control

Workflows

One way to ensure reproducibility is to have a well-organized project workflow and management. As you may have already noticed, in research there are many different files - you may have code, CSV files with your data, documents of proposals and reports, drafts with different versions, comments from peers or supervisors etc. A common way to organize your files is by **project - a project-oriented workflow**. Here, each project that you are working on is **in its own self-contained folder**, with its **different data types in different folders**. I.e. your raw data should be in a different folder than your processed data, documents and files should be in another folder etc.



Note the differences between the first pane and the other two. The first pane, different file types are in the same folder which makes it difficult to find and filter through projects. In the second pane, all the projects are sorted into their own respective folders and with their own subdirectories based on their analysis or function.

File Paths

Having a default file structure can also help with your own mental organization of your file paths. There are two types of file paths: **Absolute and relative file paths**

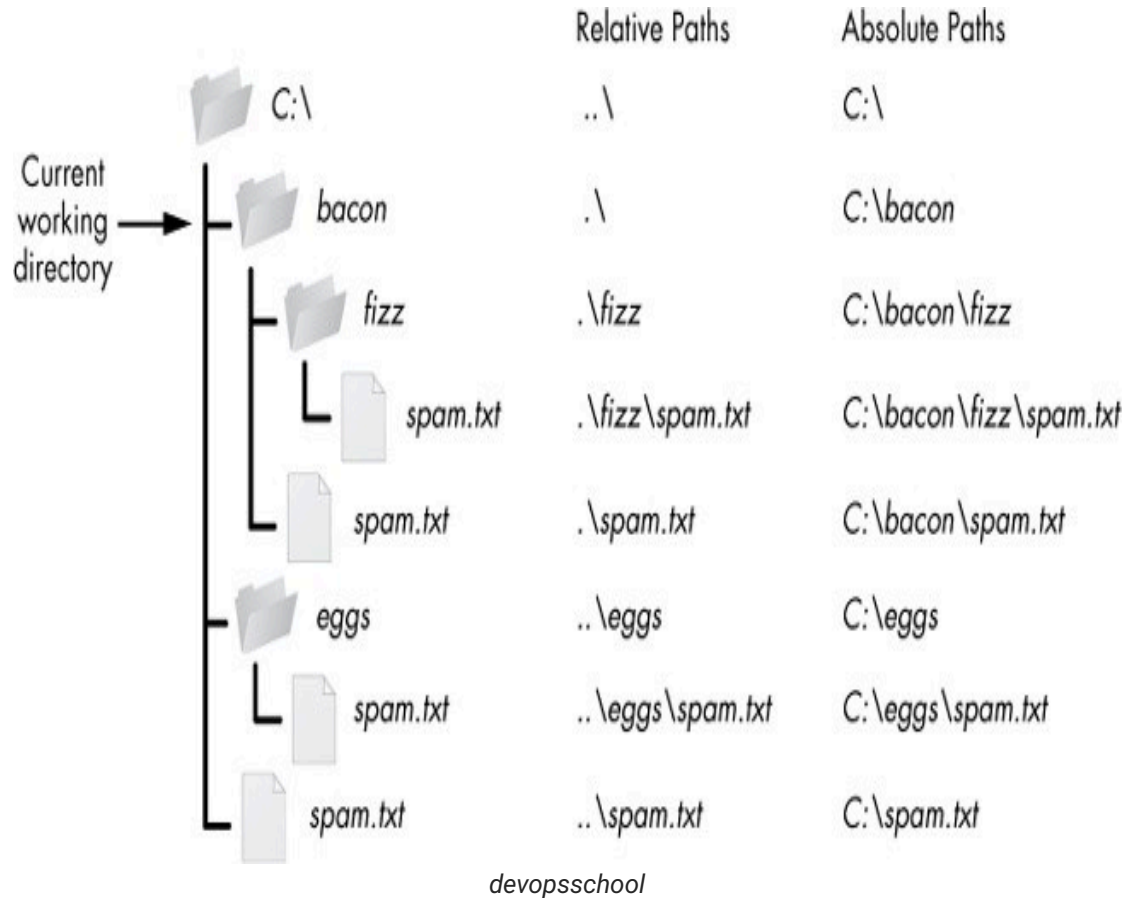
Absolute File Paths

Absolute file paths **start at your root folder and mention every directory before arriving at your file**. For example, if I wanted to get the absolute file path of the CSV in the above figure, it would look something like this: `/Users/<username>/Desktop/Projects/Salt Tolerance/Chlamy sample - Sheet1.csv`, Where `Chlamy sample - Sheet1.csv` is the name of my file, which is located in the `Salt Tolerance/` directory. This directory is in the `Projects/` directory which is on my `Desktop/` (also a directory), within my user folder (`<username>`), within the `Users/` folder in the **root** (`/`). The `/` **denotes a directory (or also called a folder)**. Absolute file paths can be very useful, however, **they are very sensitive to breaking**. For example, if I moved `Projects/` to `Documents/` rather than `Desktop/` my path would **no longer work and cause an error**.

Relative File Paths

Relative file paths are **relative to the current directory that you are working in**. Your current directory will be noted with a `./` and the parent directory will be denoted with `../` For example if I am in the `Figures/` directory and wish to go to the `References/` directory I can use the relative file path: `../References/`. The benefit of using absolute paths is that if anything happened with `Projects/` (eg: I moved it to `Documents/` from `Desktop/`) **my path would still work**.

There is no right or wrong file path format! Choose the one that works for you and the type of document that you have. If you know the hierarchy of your directories, relative paths might serve better. However, if you do not know the hierarchy but know where your file is in general then an absolute path might serve better.



Just like variable names, **it helps to name your folders something meaningful.** Folders called `1/` and `2/` do not do much to help you when you need to find your files.

bash

Before we dive into git, let's refresh ourselves with some commonly used bash commands. bash is a common Unix shell and command language. Since accessibility to bash is dependent on your operating system, we will run it within CoCalc (View > Terminal). For your note-keeping, we can use bash within a Python notebook! We just need to tell Python, hey! this cell is just going to run BASH - don't run me with Python! **We can do this by using a magic command denoted with a %%** . By running `%%bash` , we're telling Python to run this entire cell in BASH. **The magic command must be the first line of the cell.** Let's try it out:

(Note in R, you can also run command line commands using `system()`)

In [0]: `%%bash`

Amazing! Now let's go ahead and play with some commonly used commands:

In [0]: `%%bash`
`pwd`

In [0]:

```

%%bash
# that was the magic command
pwd # path to working directory. Absolute file path
# ls # list command. This will list all the files in your current folder
# ls -l # long list. Gives more information of the file type and data
mkdir temp/ # make directory.
cd temp/ # change directory. This "moves you" into the folder of interest
ls -l
pwd

touch hello.txt # creates the file hello.txt
# vim hello.txt
# to get to insert mode type "i"
# to exit insert mode, hit esc
# have to type :wq, w will save, q will exit
# instead of vim, you can also use any text editor of your choice (eg: nano,
emacs)

less hello.txt # view the file contents. To exit hit q

cat hello.txt # print the contents of the file to my screen

mv hello.txt ../ # move. It will move hello.txt into the parent folder

mv hello.txt ./temp/hello_there.txt # mv can also rename files

cp hello_there.txt hello_there_copy.txt # copy. Will copy the file, provide
its new name

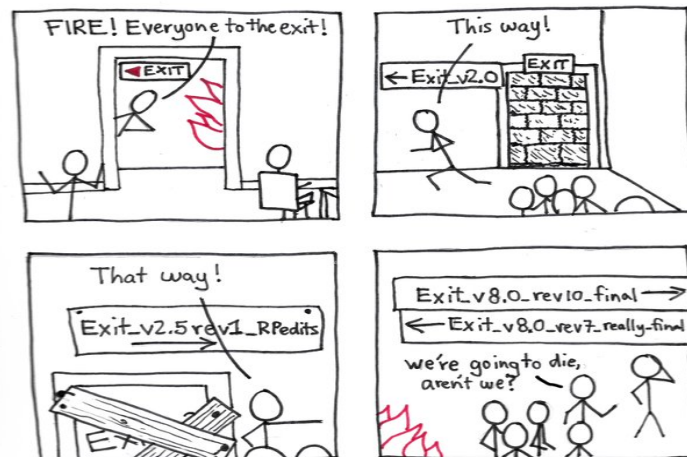
rm hello_there.txt # remove. Deletes the file

rm -r # deletes a directory or folder pls use with caution

```

Git

Now for the program of the hour! Git is a version control system that can be used to track file changes. While most commonly thought to be used for coding files only, **it can also be used for any file type** including documents, figures, presentations etc. It's a great way to edit your files but also be able to **revert to a previous version if the need arises**. Used in tandem with **GitHub**, it can be a great way to **easily collaborate on projects with others**. It is important to note that you can use Git without having a GitHub account, however, you cannot use GitHub without Git.



Stop, Drop, and Use a Versioning System
@redpen blackpen

Configure git global settings on your local computer

Before we start we need to tell git who we are so that we know who is making the files. We can do this using the `git config` commands

```
In [0]: %%bash

# Tell git your email. Use your github email
git config --global user.email "<your email>"

# Next tell git your name
git config --global user.name "<username>"

# Change the name of your first branch from "Master" to "main"
git config --global init.defaultBranch main
```

```
In [0]: %%bash
git config --get user.name
```

Local git repo

Let's make a local git repository. A repository (**or a repo for short**) is a storage area for a project containing all the files for the project and the history of the changes made to it. **This repo will be on your local computer (In this case, your CoCalc account)**

```
In [0]: %%bash

mkdir week7_git_lesson
cd week7_git_lesson
ls
```

```
In [0]: %%bash
# Initialize the git repository
cd week7_git_lesson

git init
```

```
In [0]: %%bash
cd week7_git_lesson

# Now that we have our git repo, lets add a file to it
touch first_file.txt
ls

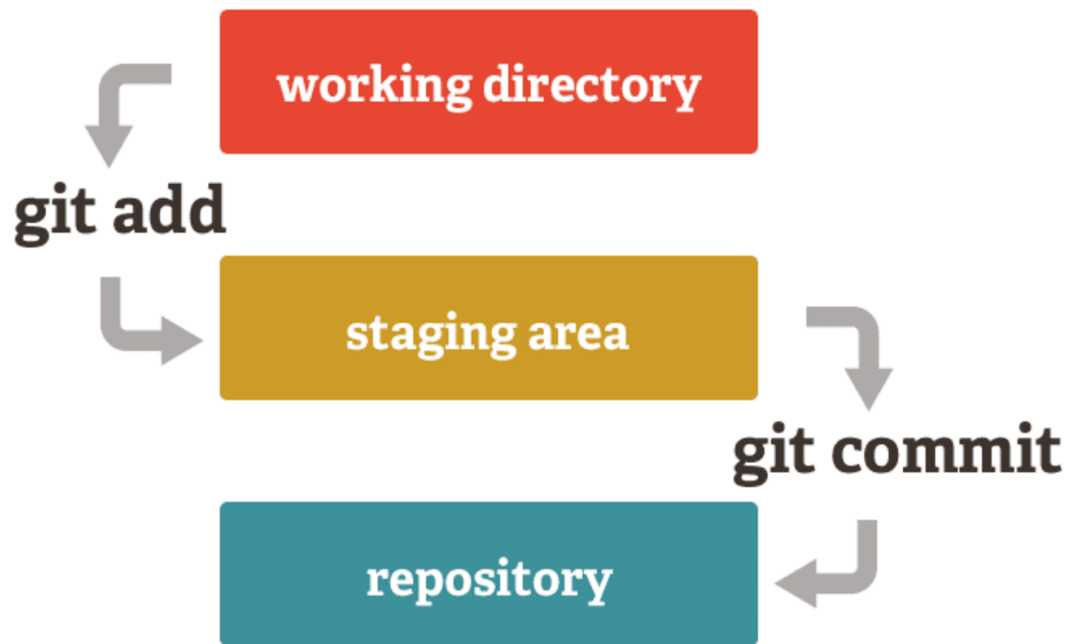
# Let's see how git is doing by checking it's status
git status

# git has noticed some untracked changes
```

Hold your horses! The staging environment, the commit and you

Before we get going on our git train, it's important to differentiate between your working directory, the staging environment and a commit. Commits are **records of the changes you've made since your last commit**. These are what allow you to jump between the different stages of your project. However, git does not commit on its own, **you need to tell git what you want to commit**. To do this we first need to **adding the file to the staging area** via

the command `git add`. Once you've made a nice package of files in the staging area, you **can then commit your changes to your repo** in the `.git` repository. Let's see what this looks like



```
In [0]: %%bash
cd week7_git_lesson

# Now let's add our file to the staging environment
git add first_file.txt

git status # We see now that our file isn't committed but it is about to be
```

```
In [0]: %%bash
cd week7_git_lesson

# Now let's commit our file. We have the option to also add a message using
the -m flag. It is always beneficial to write something useful so that when
you are looking back between your versions it is easy to find what you're
looking for
git commit -m "Created empty file called first_file.txt"

git status # We see now that our file isn't committed but it is about to be
```

```
In [0]: %%bash
cd week7_git_lesson

# Great! You just created your first commit! Let's see what happens when we
edit our file to add some information to it!
vim first_file.txt

git status #see that the file has been modified

# what was different from the last commit?
git diff first_file.txt

git add first_file.txt #stage our changes
```

```
git status
```

```
git commit -m "Added some filler text to file"
git status
```

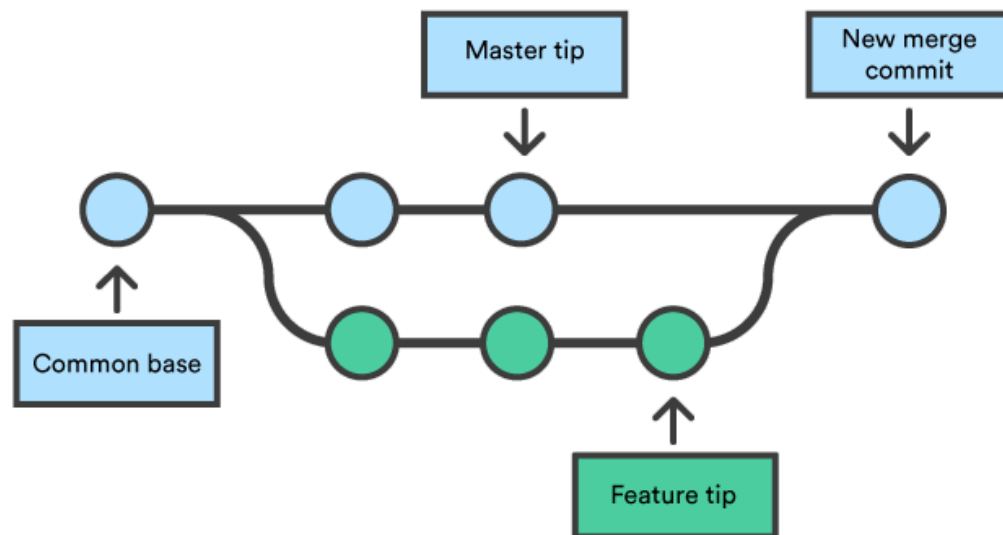
In [0]:

```
%%bash
cd week7_git_lesson

#let's see what we've done so far
git log
```

Branching in Git

Branching is a great way to work on a new idea, bug fixes or developing features **without affecting your main/master branch**. They are designed to be **temporary** and to facilitate edits and trials without breaking your original codebase. They allow you to keep your original `main` branch as is. For example, say I had a website and I wanted to make a new webpage. I can branch off of my main branch, create my webpage and then merge once it is completed. This way my website still stays functional as I work on my new webpage.



There are two phases of branch creation: **1) Making the branch** and **2) Moving to that branch**. Both of these steps can be combined into one but let us take it one at a time. First, let's make a new branch called `new_files`. Just like commits and variable names, it helps to have meaningful branch names so that you know what that branch is meant to be working on

In [0]:

```
%%bash
cd week7_git_lesson

git branch new_files
git log
git status # notice that while we made a branch our status has not changed
git branch --list

# move to that branch
git checkout new_files
```

```
git branch --list

git log
git status # now our branch has changed
```

In [0]:

```
%%bash
cd week7_git_lesson

# Let's make some changes to our files

# edit first_file.txt

cp first_file.txt second_file.txt # make a copy called second_file.txt
git status

# edit the second_file.txt

git commit -a -m "Added another line of text about the weather to file"
#Notice how it only added changes and committed for the first_file.txt as
git was watching it

# Let's add and commit the other files
git add *
git commit -m "adding second file"

# now let's merge our branch back to the main one
# First head back to the main branch
git checkout main
ls

git branch --list

# Merge your branch
git merge new_files

# Delete the old branch since there's no use for it anymore
git branch -d new_files

git status
```

Going back to a previous commit

Sometimes we had versions of our code that worked better or documents that were worded more eloquently, but we no longer have them as we've modified the same file. With Git, **as long as the file was committed** we can revert to that version. Let's play around with it:

In [0]:

```
%%bash
cd week7_git_lesson

#make a temporary file and commit it
touch temp.txt
git add temp.txt
git commit -m "adding temporary file temp.txt to delete"

# Remove the file and update git
rm temp.txt
git status # while we deleted the file locally, it is still in our repo
git rm temp.txt
git commit -m "deleted temporary file temp.txt"

#make some minor changes
echo temp2 > temp2.txt
```

```

git add temp2.txt
git commit -m "added temp2.txt"

# Revert back to the previous commit
# get the commit ID
git log
git revert <commit_id>
ls
git commit -m "reverting back to other commit with temp.txt"

# Revert but keep changes, can also do with the one above ^ however this one
will delete the commits following the one you want to go to
touch temp3.txt
git add temp3.txt
git commit -m "added temp3.txt"
rm temp3.txt
git restore temp3.txt
git commit -m "message"

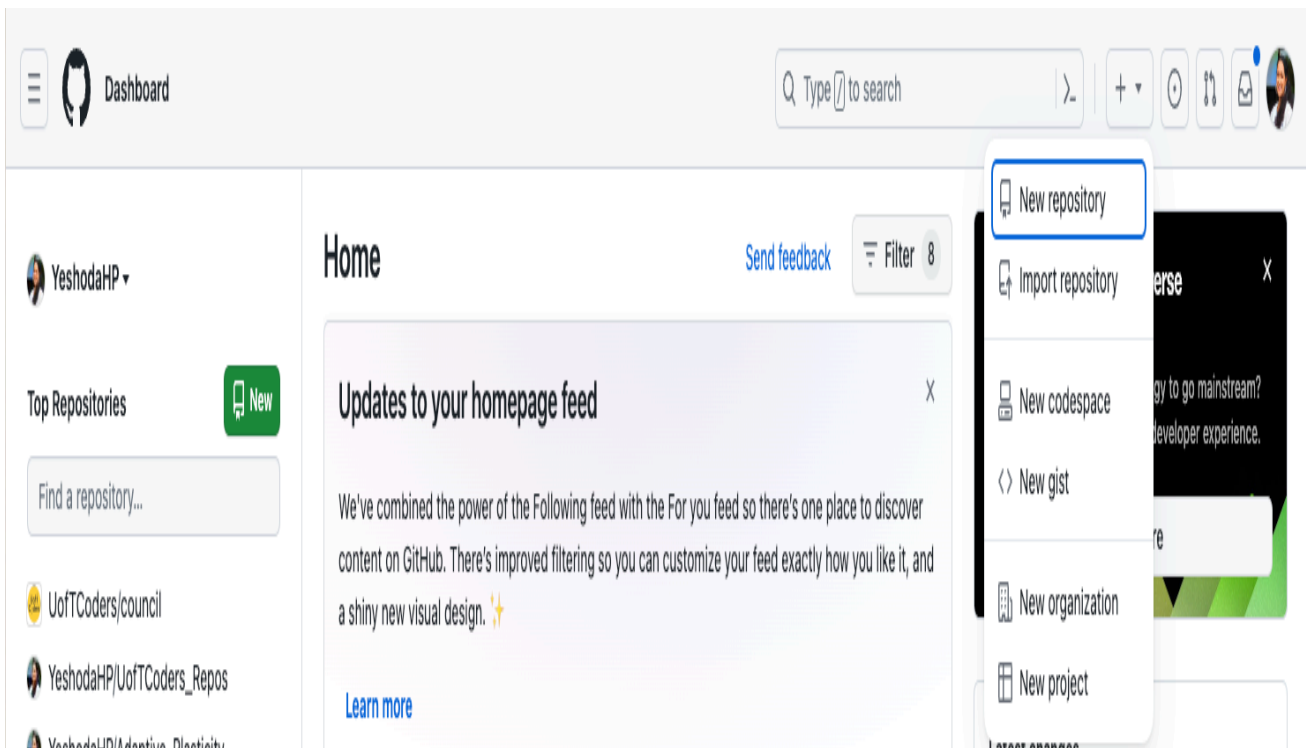
# Other options
git checkout <commit_id>
git checkout main # return to original

git reset --hard <commit_id>

```

Integrating GitHub

Now that we have some of the basics under our belt let's push something to GitHub. First we need to create a new repo on GitHub. We can do this by logging in, pressing the plus button (create new) and selecting "New Repository"



Next we can go ahead and name our repository and add a brief description. Once that is done go ahead and press "Create repository".

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *

 YeshodaHP

Repository name *

BINF5503_Week10

✓ BINF5503_Week10 is available.

Great repository names are short and memorable. Need inspiration? How about [expert-octo-train](#) ?

Description (optional)

Repo for Week10 Module5 class on reproducibility

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

 You are creating a public repository in your personal account.

Create repository

GitHub gives us the option of pushing an existing repo from the commandline. Let's go ahead and push our week7_git_lesson folder to our GitHub:

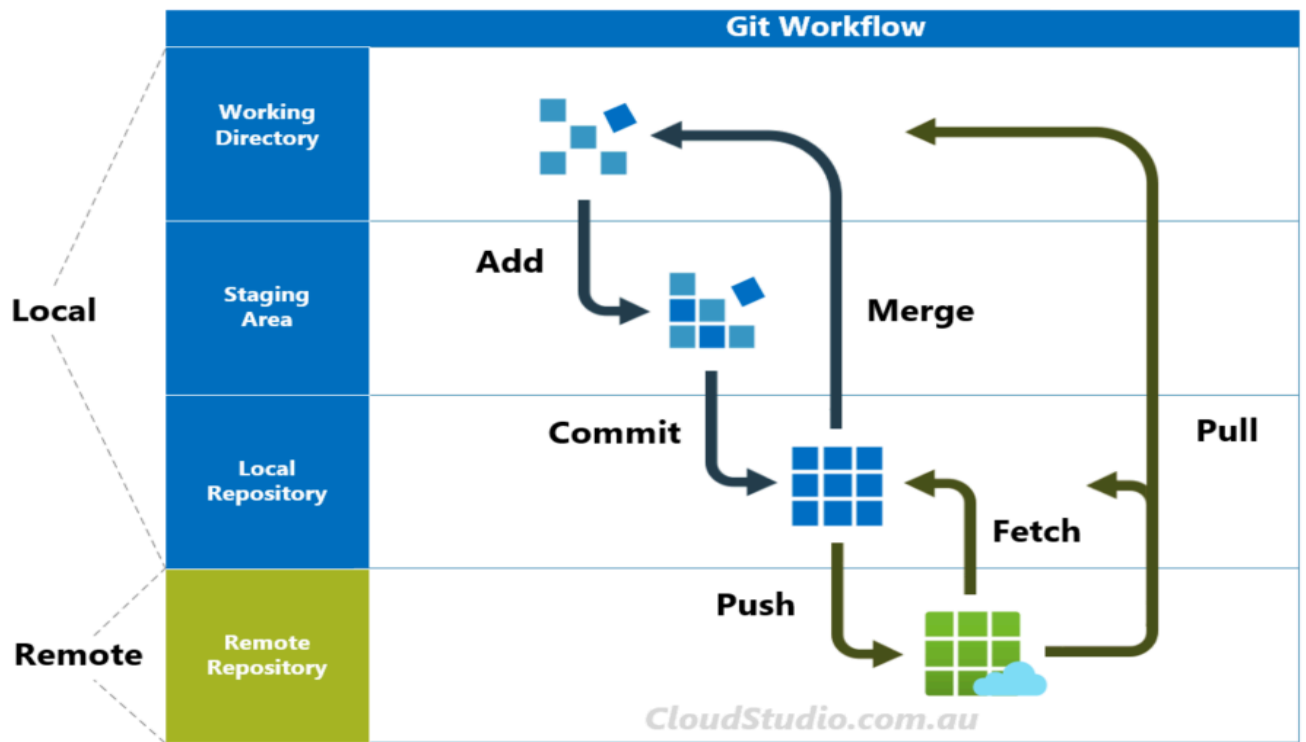
In [0]:

```
%%bash
cd week7_git_lesson

git remote add origin https://github.com/<username>/<repository>.git
git branch -M main
git push -u origin main

# This will prompt you to input your username and password (Now using token)
# To generate a token go to settings > developer settings > Personal access
tokens > Generate new token
```

Great! Now we can see our files and commits on our GitHub! Note that now when we want to place our files onto GitHub (the remote directory), we now have to `push` it there. Let's try creating a file and pushing it to our GitHub



```
In [0]: %%bash
cd week7_git_lesson

#Make a new file, add and commit
touch github.txt
git add github.txt
git commit -m "Adding new github.txt file"

# now push it to GitHub
git push
```

Collaborative Repos

Great! Now that we've pushed files to GitHub let's try to use it in a collaborative way. There are two common collaborative workflows:

1. The Shared Repository Workflow

- Assigned collaborators work on their own branch and then merge into the main

2. The Fork and Pull Workflow

- Collaborators fork the repository (make a personal copy) and work on their own fork. Merging requires pull requests. The project lead will accept pull requests (PRs), review and accept changes made

Let's see how the shared repo workflow looks!

Shared Repo

Let's try creating a file with your name as the file name, containing the name or hexcode of your favourite colour. Let's push it to this class's GitHub repository. First, we will need to clone the repo located at:

https://github.com/Sabrina-Ge/BINF5503_2025Winter.git. Next, create a branch with your name, create your file, commit and push!

Remember to always pull before you start to do your work so that you're on the most recent branch!

```
In [0]: %%bash
#clone the repo
git clone https://github.com/Sabrina-Ge/BINF5503_2025Winter.git

cd Lecture/

# fetch new work done by others - it does not merge with your branch
git fetch

# merge the work you've done with others
git merge

# will pull all the online work and merge it with your local work. Make sure
you commit your own work before pulling
git pull

# Create a branch with your name as the branch and add a new file with your
name and your favourite colour!
git branch {Write your name}
git checkout {Branch name}
# vim {Your Name}.txt
git add {Your Name}.txt
git commit -m "{YOU} are adding your file"
git push --set-upstream origin {Branch Name}

git push origin {Branch Name}

# go back to the main branch
git checkout main

# delete your branch when it has been fully merged
git branch -d {Branch Name}
```

Recap

Amazing Job! So far today we learned about :

- Reproducibility and why it is important
- File hierarchies
- The BASH console and some common commands
- An intro to Git
 - Where we learned to make a local repository
 - How to commit changes
 - Create branches
 - How to revert to previous commits
- GitHub:

- How to connect and clone
- How to collaborate with other users.

We did a lot of Git today, and we know there is a lot of jargon surrounding git so here is a brief recap:

Term	Description
Add	Adding your changes to the “Staging” area
Branch	A parallel version of a repository where changes can be made that do not affect the main repository. New branches are often used to test changes or new ideas, which can later be merged with the base branch
Checkout	Moving from one branch to another
Clone	Make a copy of the your GitHub repo on your local computer
Commit	Takes the files in your staging area and store their changes permanently in your Git directory
Head	Head is a reference or pointer of the latest commit on your branch, there can only be one head at a time. You can move the head to a different commit by using checkout - this is known as a detached head
Merge	Incorporate changes into the branch you are on
Pull	A combination of fetch and merge. This fetches the files from the remote repo and merges them with your local files
Pull Request	Term used in collaboration. You “issue a pull request” to the owner of the upstream repo asking them to pull your changes into their repo (accept your work)
Push	Sync your file changes to the remote repo on GitHub

Make sure to also check out the official Github cheatsheet (a copy is saved on Blackboard). Happy coding!